2일차 강의 자료 · 8시간

Cursor AI를 활용한 Dual-Track TDD 실전

UI RED/GREEN · Logic RED/GREEN · REFACTOR · Traceability · Prompting BP

1교시	To-Do 리스트 생성 요청 프롬프트 실습	Cursor AI 협업 전략 완성
2교시	PHASE 1 — [TRACK A] UI/Boundary RED	UX Contract → 실패 UI 테스트 선언
3교시	PHASE 1 — [TRACK B] Logic RED	도메인 RED + Gherkin → pytest 변환
4교시	PHASE 2 — [TRACK A] UI GREEN	최소 UX 구현 + 로직 주입
5교시	PHASE 2 — [TRACK B] Logic GREEN	최소 도메인 구현 (Cursor Agent)
6교시	PHASE 3 — 통합 REFACTOR	Lock-Step 리팩토링 + SOLID 적용
7교시	TRACEABILITY — 4계층 추적성 검증	Invariant→UI Contract→Domain→Unit
8교시	Prompting Best Practices 발표	BP 정리 & 팀 공유 & 회고

2일차 시간표 (8시간)

일정

1교시	60분	To-Do 리스트 생성 요청 프롬프트 실습	Work Order 구조 · Cursor Agent 전략 · 실습 과제 TDD 시작
2교시	60분	PHASE 1 — [TRACK A] UI/Boundary RED	UX Contract 선언 · React Testing Library · 실패 UI 테스트
3교시	60분	PHASE 1 — [TRACK B] Logic RED	도메인 RED · Gherkin→pytest · 실패 확인
4교시	60분	PHASE 2 — [TRACK A] UI GREEN	최소 UX 구현 · 로직 주입 원칙 · Cursor Agent 활용
// 점심 휴식 60분			
5교시	60분	PHASE 2 — [TRACK B] Logic GREEN	최소 도메인 구현 · YAGNI · 하드코딩 허용
6교시	60분	PHASE 3 — 통합 REFACTOR	Lock-Step 리팩토링 · SOLID · 코드 스멜 제거
7교시	60분	TRACEABILITY — 4계층 추적성 검증	Invariant→UI Contract→Domain Test→Unit
8교시	60분	Prompting Best Practices 발표	BP 팀 발표 · KPT 회고 · SMART 목표

01

To-Do 리스트 생성 요청 프롬프트 실습

Work Order 구조 · Cursor AI 협업 전략 · TDD 착수 점검

1교시

60분

1.1 To-Do → Work Order — Cursor AI 협업의 핵심 구조

1교시

핵심 전제: "모든 to-do는 테스트로 변환되지 않는다 — 판단(Decision)을 포함한 to-do만 변환된다"

To-Do List *"무엇을 할 것인가"*

- [ ] Magic Square Entity 정의
- [ ] SquareValidator 구현
- [ ] 빈칸 탐색 알고리즘
- [ ] UI 격자 표시
- [ ] 결과 출력

Work Order *"어떻게, 검증은"*

```
WI-001: MagicSquare(size) → ECB <<entity>>
WI-002: is_valid(grid)->bool, 합≠34→False
WI-003: find_missing()->list, 빈칸0개→raise
WI-004: GridUI <<boundary>>, 4×4 테이블
WI-005: ResultDisplay, 해/실패 메시지
```

To-Do : Work Order = 1 : N · 하나의 할 일 → 여러 세부 작업 지시 · Work Order = 검증 가능한 계약

1.2 To-Do → Test 변환 핵심 규칙

8교시

Rule 0 — **전제 선언**
판단 포함 to-do만 변환

Rule 1 — **행동형 폐기·분해**
"로그인 화면 만들기" → "언제 거부?"로

Rule 2 — **상태 조건 필수**
"언제(When)" 없으면 테스트 불가

Rule 3 — **Scenario 매핑**
L0~L3 중 하나에 속해야 함

Rule 4 — **UX/Logic 분리**
"에러 보임(UX)" + "제출 거부(Logic)"

Rule 5 — **언어 제한**
보인다/허용한다 → 변환 가능

Rule 6 — **1:1 대응**
to-do 1개 = 테스트 1개만

Rule 7 — **이름 = 문장**
테스트명 = to-do 문장 그대로

Rule 8 — **실패 의미 선언**
실패 = 개념·책임 위반 가능성

Rule 9 — **생명주기 일치**
추가→RED, 완료→GREEN, 삭제→테스트삭제

1.3 체크박스 · 단계별 번호 · Requirements 추적 체계

8교시

📋 체크박스 & 단계별 번호 체계

```
## Epic-001: 마방진 검증 시스템

### US-001: 합 검증
- [x] TASK-001: MagicSquare Entity 정의
- [ ] TASK-002: SquareValidator 구현
  - [ ] TASK-002-1: is_valid() RED
  - [ ] TASK-002-2: is_valid() GREEN
  - [ ] TASK-002-3: REFACTOR

### US-002: 빈칸 탐색
- [ ] TASK-003: MissingFinder 구현
  * 선택: N-Queen 알고리즘 활용
  > [Checkpoint] 성능 < 100ms 검증
```

🔗 Requirements 추적 매트릭스

Task ID	Req ID	Scenario	테스트	상태
TASK-001	REQ-001	L1 Happy	test_valid_ms	✅ PASS
TASK-002	REQ-002	L3 Fail	test_invalid	🔴 RED
TASK-003	REQ-003	L2 Edge	test_missing	⬜ TODO
TASK-004	REQ-001	L1 UI	test_grid_ui	⬜ TODO

추적성: Task → Req → Scenario → Test
이 연결고리가 C2C Traceability의 실천

1.4 속성 테스트 · 검증 포인트 · 체크포인트 · 선택 항목(*)

8교시

☑ 체크박스 항목

- [ ] 미완료
- [x] 완료
- [-] 불필요(취소)

완료 시 연결 테스트 GREEN으로 자동 전환

🔢 단계별 번호

Epic → User Story → Task → Sub-Task
계층 구조로 의존성 명확화
Epic-001 > US-001 > TASK-001-1

* 선택 항목 (*)

- * 선택: 알고리즘 최적화
비필수 기능을 명확히 구분
스프린트 범위 관리에 필수

> 체크포인트

> [Checkpoint] 성능 < 100ms
> [Checkpoint] 커버리지 80%+
특정 기준 달성 시 다음 단계 진행

🧪 속성 테스트

```
@given(valid_grid)
def test_valid_range(grid):
  assert all(1<=v<=16 for row ...)
```

랜덤 입력으로 속성 불변성 검증

✅ 검증 포인트

[Validate] is_valid() → True
[Validate] 커버리지 80%+
[Validate] 성능 < 100ms
각 Task 완료 기준 명시

"to-do를 지우면 반드시 테스트 하나가 사라져야 한다 — 이것이 C2C Traceability의 핵심이다"

1.5 Magic Square — 완성 To-Do 구조 예시

8교시 실습

이 구조가 To-Do → Scenario → Test → Code의 C2C Traceability 완전 구현 예시입니다

```
## Epic-001: Magic Square 4×4 완성 시스템

### US-001: 마방진 유효성 검증 (REQ-001)
- [x] TASK-001: MagicSquare Entity 정의 (<<entity>>) → test_entity_creation PASS
- [ ] TASK-002: SquareValidator.is_valid() 구현 (<<control>>)
  - [ ] TASK-002-1: [RED] test_invalid_magic_square() 작성
  - [ ] TASK-002-2: [GREEN] 최소 구현으로 통과
  - [ ] TASK-002-3: [REFACTOR] 타입힌트 + 독스트링
  > [Checkpoint] test_invalid_magic_square 통과 확인 후 진행

### US-002: 빈칸 위치 탐색 (REQ-002)
- [ ] TASK-003: MissingFinder.find_missing() 구현 (<<control>>)
  - [ ] TASK-003-1: [RED] test_missing_not_found() 작성 (L3 Failure)
  - [ ] TASK-003-2: [GREEN] 빈칸 탐색 최소 구현
  * 선택: Backtracking 최적화 알고리즘
  > [Checkpoint] 성능 < 100ms 검증 · [Validate] 커버리지 80%+
```

2일차 강의 자료 · 8시간

워크 북 실습

D2_1.실습- Cursor AI To-Do 리스트 생성 요청

여기까지 작업한 다음 깃헙 저장소를 만들고 깃헙에 업로드합니다.

1.3 2일차 TDD 착수 체크리스트 & 실습 과제 확인

1교시 실습

☑ 착수 전 필수 확인

- ☐ .cursorrules 파일 설정 완료
- ☐ ECB 분석표 작성 완료 (어제 4교시)
- ☐ Gherkin 시나리오 최소 3개 준비
- ☐ UX Contract 판단 언어 결정
- ☐ Logic Rule 판단 동사 결정
- ☐ GitHub Repository 생성 완료
- ☐ dev 브랜치 생성 및 전환
- ☐ pytest, pytest-cov 설치 확인

📋 오늘 실습 산출물 목표

PHASE 1 RED

- UI RED 테스트 파일 (test_ui_magic.py)
- Logic RED 테스트 파일 (test_logic_magic.py)

PHASE 2 GREEN

- UI GREEN 구현 (grid_ui.py)
- Logic GREEN 구현 (magic_square.py)

PHASE 3 REFACTOR

- SOLID 리팩토링 (REFACTOR commit)
- 커버리지 80%+ (pytest-cov 확인)

02

클린코드 핵심 원칙

Clean Code - 읽기 쉽고 유지보수 가능한 코드를 작성하는 법

2교시

60분

왜 클린코드인가? - ROI 분석

초기 비용 vs 장기 이익

+20% 초기 작성 시간	**필요** 리팩토링 노력	**-60%** 버그 감소
-50% 유지보수 시간	**-40%** 온보딩 시간	**-30%** 코드 리뷰 시간

클린코드 5대 원칙

① 명확한 이름

- 변수/함수명만 봐도 의미를 알 수 있어야 한다
- 약어 지양, 도메인 언어 사용
- 예: calc(o,c) → calculate_discounted_price()

② 중복 제거 (DRY)

- Don't Repeat Yourself
- 같은 코드가 2곳 이상 → 함수로 추출
- 유지보수 변경 포인트를 1곳으로

③ 단일 책임 (SRP)

- 하나의 함수/클래스는 하나의 일만
- 변경 이유가 오직 하나여야
- OrderService._validate_user() 분리

④ 작은 함수

- 20줄 이하 권장
- 들여쓰기 깊이 2-3단계 이하
- 파라미터 3개 이하 권장

⑤ 예외 처리

- 에러를 숨기지 말고 명시적 예외 raise
- 커스텀 예외 클래스 활용
- FastAPI: HTTPException으로 변환

⑥ 타입 힌트

- Python 3.x: def func(x: int) -> str
- IDE 자동완성, 문서화 효과
- mypy로 정적 타입 검사 자동화

원칙 ① 명확한 이름 - Before/After

❌ **Bad Code**

```python
def calc(o, c):
    d = o * c.pct
    return o - d
```

✅ **Good Code**

```python
def calculate_discounted_price(
    original_price: float,
    coupon: Coupon
) -> float:
    discount_amount = (
        original_price
        * coupon.discount_percentage
    )
    return original_price - discount_amount
```

원칙 ③ 단일 책임(SRP) - Before/After

❌ **Bad Code**

```
def process_order(user_id, items,
                  coupon_code):
    # 사용자 검증
    user = db.query(User).filter_by(
        id=user_id).first()
    # 쿠폰 검증 + 가격 계산
    # 주문 저장 + 알림 전송
    # 한 함수가 너무 많은 책임!
    ...
```

✅ **Good Code**

```
class OrderService:
  def create_order(self, user_id, items,
                   coupon_code) -> Order:
      user = self._validate_user(user_id)
      coupon = self._get_coupon(coupon_code)
      total = self._calculate_total(
          items, coupon)
      order = self._save_order(user,
          items, total)
      self._notify_user(user, order)
      return order
```

모듈화 품질 지표 - 결합도와 응집도

구분	종류	설명	품질
결합도 (낮을수록 좋음)	내용 결합도	내부 기능/자료 직접 참조	최하
	공통 결합도	전역변수 공유	하
	자료 결합도	인터페이스 자료요소로만 구성	최상
응집도 (높을수록 좋음)	기능적 응집도	단일 기능 수행	최상
	순차적 응집도	출력 → 다음 입력으로 연결	상
	우연적 응집도	연관성 없는 요소	최하

객체지향 설계 SOLID 5대 원칙

S - 단일 책임 (SRP)

- 하나의 클래스는 하나의 책임
- 변경 이유가 하나여야 한다
- God Class 분리

O - 개방/폐쇄 (OCP)

- 확장에는 열려있고 수정에는 닫힘
- Strategy Pattern 활용
- 새 결제 수단 추가 시 기존 코드 무수정

L - 리스코프 치환 (LSP)

- 자식 클래스로 부모 대체 가능
- Bird→Penguin fly() 위반 사례
- Flyable 인터페이스로 분리

I - 인터페이스 분리 (ISP)

- 클라이언트는 불필요한 메서드 의존 X
- BasicAnimalActions + Flyable + Swimmable
- Dog는 fly/swim 불필요

D - 의존성 역전 (DIP)

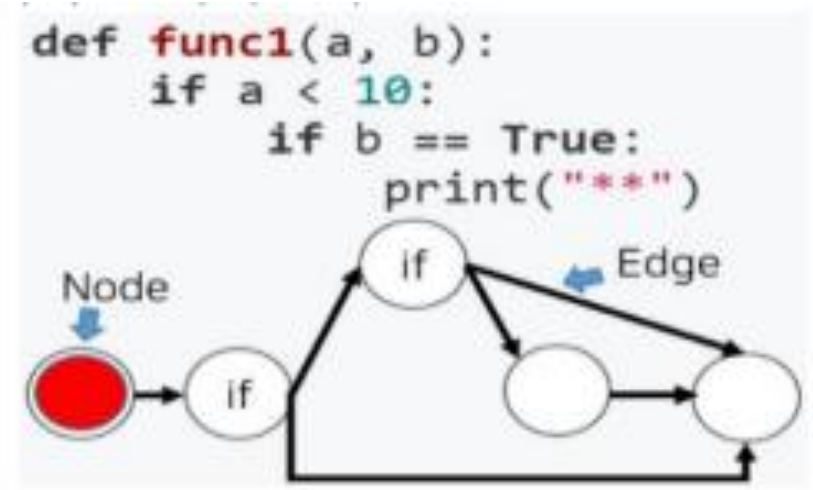
- 구체 클래스가 아닌 추상에 의존
- MessageService ABC ← Email/SMS
- 상위 모듈이 하위 세부에 의존 X

클린코드 커밋 전 체크리스트

- ✅ 변수/함수명이 의미를 명확히 전달하는가?
- ✅ 중복 코드가 제거되었는가? (DRY)
- ✅ 함수가 한 가지 일만 하는가? (SRP)
- ✅ 함수 길이가 20줄을 넘지 않는가?
- ✅ 들여쓰기 깊이가 3단계 이하인가?
- ✅ 주석 없이도 코드 의도를 이해할 수 있는가?
- ✅ 예외 처리가 적절하게 되어있는가?
- ✅ 타입 힌트가 명시되어 있는가?
- ✅ McCabe 복잡도(Cyclomatic Complexity)가 10 이하인가?

클린코드 커밋 전 체크리스트

- **McCabe's Cyclomatic Complexity복잡도**
- **예시**



$$V(G) = E - N + 2P$$

- $V(G)$: Cyclomatic 복잡도
- E: 엣지(Edges) 수 – 흐름선
- N: 노드(Nodes) 수 – 실행 지점
- P: 독립적인 그래프의 개수 (보통 1)

- **복잡도 $V(G) = E - N + 2P$ 공식으로 E 는 그래프의 엣지, N는 노드, P는 연결되어 있지 않은 그래프의 부분 수로 계산한다. 따라서 위 그림의 코드 그래프는 6(Edge) – 5(Node) + 2 = 3 으로 계산된다.**
- **복잡도 3은 이 함수에 대해 세 가지 경로(테스트 케이스)가 필요함을 뜻한다.코드가 복잡할수록 이 값은 높아지고, 이는 리팩토링 또는 함수 분리가 필요할 수 있음을 암시한다.**

02

테스트 기반 개발

Test Theory - 왜, 무엇을, 어떻게 테스트하는가

2교시

60분

테스트를 작성하는 진짜 이유

❌ 잘못된 오해

- 버그를 찾기 위해
- QA가 시키니까
- 커버리지를 채우기 위해
- 배포 전 확인용으로만

✅ 진짜 이유

- 설계를 검증하기 위해
- 리팩토링의 안전망으로
- 살아있는 문서로 활용
- 빠른 피드백 루프 확보
- 회귀(Regression) 방지

회귀(Regression)란 무엇인가?

이미 잘 동작하던 기능이
수정 이후에 다시 깨지는 현상

Testing 방법

변경 후 기도하기(Edit and Pray)

- 신중한 검토 -> 변경 대상 이해 -> 변경 -> 정상 동작 확인-> 의도하지 않은 변경 여부 확인(Side effect)
- 전문적인 방법처럼 보이지만, 신중 ≠안정

체계적인 보호 없이 변경을 시도하는 방식으로, 위험성이 높음

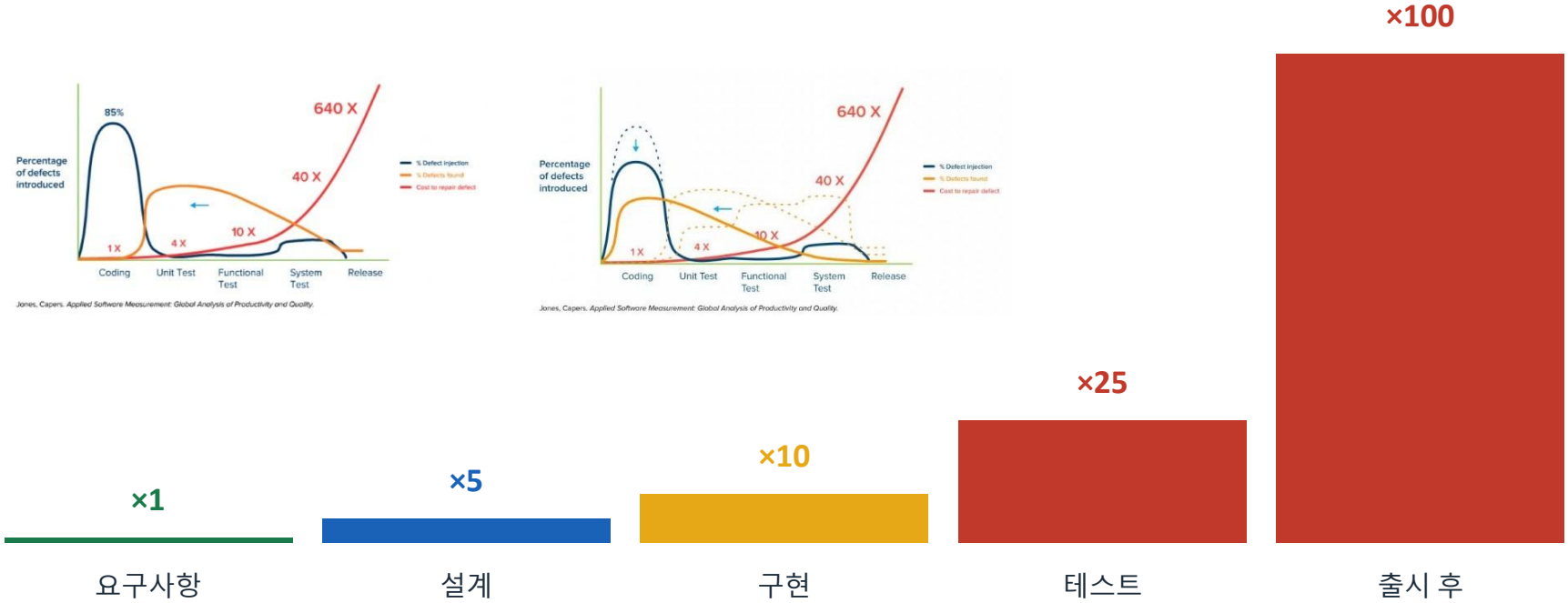
보호 후 수정하기(Cover and Modify)

- 테스트 루틴으로 코드를 보호
- 문제 발생시 빠르게 찾을 수 있음
- 테스트 루틴 = 피드백

코드 변경 후 문제를 사전에 감지하여, Side Effect를 방지하고 안전하게 유지보수하는 방식

결함 비용 - Shift-Left Testing

결함 발견 시점이 늦을수록 수정 비용이 기하급수적으로 증가→ 초기 테스트(Shift-Left)가 핵심



출처: https://www.stickyminds.com/article/shift-left-approach-software-testing

테스트 종류 분류

화이트박스 테스트

- 내부 로직을 보면서 수행
- 구조 테스트, 루프 테스트
- 개발자 관점

블랙박스 테스트

- 외부 명세 기반, 기능 테스트
- 동등분할, 경계값 분석
- 사용자 관점

동적 테스트

- 프로그램 실행이 필요
- Unit, Integration, E2E
- 실제 동작 검증

정적 테스트

- 실행 없이 구조 분석
- 코드 검사, 워크스루
- lint, mypy, bandit

기능 테스트

- Unit Testing
- Integration Testing
- System Testing, Regression

비기능 테스트

- Performance, Load, Stress
- Security, Usability
- Reliability, Compatibility

테스트 피라미드 - 전략적 분배

E2E Tests

적음 / 느림 / 비쌈 / 전체 흐름 검증

Integration Tests

중간 / API+DB 통합 테스트

Unit Tests

많음 / 빠름 / 저렴 / 함수 단위 검증

TDD 효과: 개발시간 15~35% 증가 → 결함률 40~90% 감소 (출처: Microsoft)

좋은 테스트의 특징 - F.I.R.S.T

원칙	의미	설명
F - Fast	빠르게 실행	단위 테스트는 밀리초 단위, 자주 실행 가능해야
I - Isolated	독립적 실행	다른 테스트에 의존하지 않고 독립 실행
R - Repeatable	반복 가능	언제 어디서나 동일한 결과 보장
S - Self-validating	자가 검증	자동으로 성공/실패 판단 (assert)
T - Timely	적시 작성	코드 작성과 함께(또는 이전에) 테스트 작성

테스트 구조 - Given/When/Then (BDD)

❌ **Bad Code**

```
# ❌ 구조 없는 테스트
def test_order():
    user = User(id=1)
    order = create_order(1, items)
    assert order.total == 10000
    # 무엇을 테스트하는지 불명확
```

✅ **Good Code**

```
# ✅ Given/When/Then
def test_order_with_discount_coupon():
    # Given (준비)
    user = create_test_user()
    coupon = create_coupon(
        discount_percentage=0.1)
    # When (실행)
    order = create_order(
        user.id, items, coupon.code)
    # Then (검증)
    assert order.subtotal == 10000
    assert order.discount == 1000
    assert order.total == 9000
```

블랙박스 테스트 설계 기법

기법	설명	활용 시기
동등 분할	입력을 동등한 그룹으로 나눔 (valid/invalid)	대부분의 기본 테스트
경계값 분석	경계 지점(최솟값, 최댓값±1) 테스트	범위 조건이 있는 기능
원인-효과 그래프	입력-출력 관계를 체계적으로 분석	복잡한 조건 조합
결정 테이블	True/False 조합 경우의 수	조건이 적은 경우
상태 천이도	유효/무효 상태 전이 테스트	상태 기계 구조
오류 예측	경험과 직관으로 오류 지점 찾기	탐색적 테스트

무엇을 테스트할 것인가?

Right-BICEP 원칙

- R: 결과가 옳은가?
- B: 경계 조건이 맞는가?
- I: 역관계 확인 가능한가?
- C: 교차 검증 가능한가?
- E: 에러 조건을 강제할 수 있나?
- P: 성능 특성이 범위 내인가?

CORRECT 경계 조건

- C: 형식 일치 (Conformation)
- O: 순서 (Ordering)
- R: 범위 (Range)
- R: 참조 (Reference)
- E: 존재성 (Existence)
- C: 개체 수 (Cardinality)
- T: 시간 (Time)

PRD → 할 일 목록(To-Do) → 테스트 케이스 — C2C Traceability 흐름

"PRD는 단순한 기획 문서가 아니다 — 테스트의 원천(Source of Truth)이며, PRD의 각 시나리오가 할 일 목록을 거쳐 실행 가능한 테스트 케이스로 변환될 때 비로소 코드가 시작된다"

1 ① PRD (제품 요구사항 문서)

- Problem 정의
- 시나리오(L0~L3)
- UX Contract
- Logic Rule

→

2 ② To-Do 목록 (Work Order)

- 판단 포함 항목만
- 선별·분류
- 체크박스 구조
- 추적 번호 부여

→

3 ③ 테스트 케이스 (Test Case)

- Given/When/Then
- 실패 조건 선언
- RED → GREEN
- 자동화 검증

📌 To-Do → 테스트 변환 3대 규칙

Rule 1	판단(Decision) 포함 항목만 변환	"보인다/허용한다" → 변환 가능 · 단순 행동 to-do → 폐기
Rule 2	1 To-Do : 1 Test Case 1:1 대응	하나의 할 일 = 하나의 실패 조건 · 복합 항목은 분해
Rule 3	RED 먼저, 구현은 나중	테스트가 실패한 상태에서 시작 · FAIL 확인 후 GREEN 진행

🔍 실전 변환 예시 (Magic Square)

PRD	4×4 빈칸에 유효 숫자 입력
To-Do	[ ] 빈칸(0) 위치 두 곳을 찾는다
TestCase	find_blank_coords() 반환 좌표 검증
RED	테스트 실패 확인 후 구현 시작

무엇을 테스트할 것인가?

테스트 케이스 예

테스트 케이스			프로젝트명		인사관리 앱 시스템 구축		
			대상 시스템		정산 시스템		
단계	단위 테스트	작성자	홍길동	승인자	박문수	문서 상태	최초 작성
작성일	2020-09-01	버전	v1.0	테스트 범위	공통 모듈	테스트 조직	개발팀
테스트 ID	TC-CMM-001("Common Module" or "Core Mathematical Module") TC-AO-001 ("Arithmetic Operations")			테스트 일자	2020-09-01		
테스트 목적	사칙연산 정확도 테스트						
테스트 기능	+ 기능 점검						
입력값	1,10 / 0,1/ 0,0/ 0,-1/-1,-10						
테스트단계	테스트 케이스			예상값	중요도	성공 /실패	비고
	1+10			11	중요	성공	
	0+1			1	중요	성공	
	0 / 0			예외 발생	중요	성공	
	추가된 테스트 사례 1. 예외 처리 확인: • 0 / 0 → ArithmeticException 예외 발생 확인. 다양한 조합:						
테스트환경	자바 JDK 버전: 17 IDE: IntelliJ IDEA 2023.2 운영 체제: 윈도우 10						
전제 조건	프로그램은 오류 없이 성공적으로 컴파일되어야 합니다. 모든 종속성을 올바르게 설치하고 구성해야 합니다.						
성공 /실패 기준	성공: 모든 테스트 사례가 예상한 결과를 생성합니다. 실패: 테스트 케이스가 예상한 결과를 생성하지 않습니다.						
특별하게 필요한 절차	테스트 결과를 기록하고 이에 따라 테스트 사례 문서를 업데이트합니다. 즉각적인 해결을 위해 모든 실패를 개발팀에 전달하세요.						

1일차 강의 자료 · 8시간

워크 북 실습

D2_2.실습-Magic Square 테스트 케이스 작성

pytest - 단위 테스트 프레임워크

Pytest는 파이썬 코드를 테스트하기 위한 프레임워크로, 단위, 기능, 통합 테스트를 모두 지원하며 간단하고 직관적인 문법을 사용합니다. `test_`로 시작하는 파일 및 함수 이름을 지키면 테스트가 자동으로 탐지되고, assert 문만으로 간단하게 테스트를 작성할 수 있다는 특징이 있습니다. 또한, fixtures와 같은 강력한 확장 기능을 통해 테스트 환경 설정을 효율적으로 관리할 수 있습니다.

테스트 파일: test_로 시작하거나_test.py로 끝나는 파일에 테스트 코드를 작성합니다.
테스트 함수: test_로 시작하는 함수를 작성합니다.
테스트 코드: assert 문을 사용하여 예상 결과와 실제 결과를 비교합니다.

테스트 실행
터미널에서 프로젝트 루트 디렉토리로 이동한 후 pytest 명령어를 입력합니다.
pytest는 test_로 시작하는 파일과 함수를 자동으로 찾아 실행하고 결과를 보여줍니다.

pytest - 단위 테스트 프레임워크

❌ **Bad Code**

```
# 기본 테스트 함수
def test_add():
    assert add(1, 2) == 3

# 파라미터화 테스트
@pytest.mark.parametrize(
    "a,b,expected",
    [(1,2,3),(0,0,0),(-1,1,0)]
)
def test_add_param(a, b, expected):
    assert add(a, b) == expected
```

✅ **Good Code**

```
# Fixture 활용
@pytest.fixture
def calculator():
    return Calculator()

def test_multiply(calculator):
    assert calculator.mul(3, 4) == 12

# 커버리지 측정
# pytest --cov=app tests/
# 목표: 80% 이상

# 예외 테스트
with pytest.raises(ValueError):
    calculator.divide(1, 0)
```

테스트 더블 (Test Doubles)

Dummy

- 전달되지만 실제로 사용 안 됨
- 매개변수 채우기 용도
- 객체 목록, null 값

Stub

- 호출 시 미리 정의된 값 반환
- DB 없이 고정값 반환
- 항상 성공 응답 반환

Spy

- 실제 객체처럼 동작
- 호출 여부, 횟수 기록
- 감시(Spy) 목적

Mock

- 예상 호출을 명세
- 검증 로직 포함
- unittest.mock.patch 활용

Fake

- 실제 구현의 단순 버전
- In-Memory DB
- 빠른 단위 테스트용

1일차 강의 자료 · 8시간

워크 북 실습

D2_3.실습- 파이썬 테스트 코드 작성해보기

02

PHASE 1 — [TRACK A] UI / Boundary RED

UX Contract 선언 · UX 실패 테스트 설계 · Cursor Ask/Plan 활용

2교시

60분

2.1 UI/Boundary RED — "이 UX는 아직 구현되지 않았다" 선언

2교시

"UI는 디자인으로 RED를 만든다 — Wireframe/목업 기반 실패 테스트를 먼저 선언한다"

TRACK A — UI RED 목적

- UX Contract 언어로 테스트 선언
- 컴포넌트가 아직 없으므로 실패
- 디자인/목업 기반 검증 기준 고정
- UI 로직 포함 금지 (판단 없음)

TRACK A — UX Contract 언어

- ✅ 보인다 / 안 보인다
- ✅ 가능하다 / 불가능하다
- ✅ 활성화 / 비활성화
- ✅ 포함한다 / 포함하지 않는다

TRACK A — Cursor 모드 전략

- Ask: UX 시나리오 설계 토의
- Plan: 테스트 케이스 계획 수립
- ⚠️ Agent 사용 금지 (구현 진행 방지)
- 완료 기준: 모든 테스트 RED 확인

핵심 원칙: UI RED 단계에서는 로직 판단 코드 절대 금지 — UX Contract 언어(보인다/안 보인다)만 사용

2.2 Magic Square UI RED 실습 — UX Contract 테스트 코드

2교시 실습

UI RED 테스트 파일: test_ui_magic.py — 그리드 표시, 입력 가능성, 결과 메시지

UX Contract 테스트 (UI Track)

```python
# test_ui_magic.py
import pytest

class TestMagicSquareUI:

  def test_grid_is_displayed(self):
    """4×4 그리드가 보인다"""
    # Given: GridUI 생성
    ui = GridUI(size=4)
    # Then: 그리드가 보인다
    assert ui.is_visible() == True

  def test_blank_cell_is_editable(self):
    """빈칸 입력이 가능하다"""
    ui = GridUI(size=4)
    ui.set_cell(1,3,value=0)  # 빈칸
    assert ui.cell_editable(1,3) == True
```

추가 UX Contract 테스트

```python
def test_error_msg_hidden_initially(self):
  """초기에는 오류 메시지가 안 보인다"""
  ui = GridUI(size=4)
  # Then: 오류 메시지 안 보인다
  assert ui.error_visible() == False

def test_result_msg_hidden(self):
  """완성 전 결과 메시지 안 보인다"""
  ui = GridUI(size=4)
  assert ui.result_visible() == False

def test_submit_btn_disabled(self):
  """빈칸이 있으면 제출 버튼 비활성화"""
  ui = GridUI(size=4)
  assert ui.submit_enabled() == False
```

03

PHASE 1 — [TRACK B] Logic RED

도메인 RED · Gherkin→pytest 변환 · 실패 테스트 확정

3교시

60분

3.1 Logic RED — "이 규칙은 아직 없다" 도메인 실패 선언

3교시

"로직은 규칙으로 RED를 만든다 — 비즈니스 규칙이 아직 존재하지 않음을 테스트로 선언한다"

Gherkin 시나리오 (L0~L3)

```
Feature: Magic Square 유효성 검증

Scenario: 올바른 마방진 (L1)
  Given 모든 행/열/대각선 합=34
  When is_valid() 호출
  Then True 반환

Scenario: 합이 34 아닌 경우 (L3)
  Given 행 합이 30인 잘못된 배열
  When is_valid() 호출
  Then False 반환

Scenario: size=0 생성 (L3)
  Given size가 0
  When MagicSquare(0) 호출
  Then ValueError 발생
```

pytest 변환 (Logic Track)

```
# test_logic_magic.py
import pytest

class TestMagicSquareLogic:

  def test_valid_magic_square(self):
    sq = MagicSquare(4)  # ← 없음!
    assert sq.is_valid() == True

  def test_invalid_row_sum(self):
    sq = MagicSquare(4)
    sq.grid[0] = [1,2,3,4]  # 합≠34
    assert sq.is_valid() == False

  def test_size_zero_raises(self):
    with pytest.raises(ValueError):
        MagicSquare(0)
```

```
# pytest → 모두 FAIL → 🔴 RED 완료!
```

⚠️ Logic RED는 UI를 모른다 — UI 코드 import 금지 · 두 트랙은 완전히 독립

3.2 Logic RED — Magic Square 도메인 테스트 전체 목록

3교시 실습

총 8개 Logic RED 테스트 — Scenario L1(Happy)·L2(Edge)·L3(Failure) 모두 커버

ID	Level	테스트 이름	검증 내용	상태
T-01	L1	`test_valid_4x4_magic_square`	정상 마방진 → is_valid()==True	RED
T-02	L3	`test_invalid_row_sum`	행 합 불일치 → False	RED
T-03	L3	`test_invalid_col_sum`	열 합 불일치 → False	RED
T-04	L3	`test_invalid_diagonal`	대각선 합 불일치 → False	RED
T-05	L2	`test_missing_two_blanks`	빈칸 2개 → 좌표 리스트 반환	RED
T-06	L3	`test_missing_zero_blanks`	빈칸 0개 → MissingError	RED
T-07	L2	`test_range_1_to_16`	1~16 범위 검증 → True/False	RED
T-08	L3	`test_size_zero_raises`	size=0 → ValueError	RED

실습: 위 목록 기준으로 test_logic_magic.py 완성 → pytest 실행 → 전체 FAIL 확인 → commit(feat: logic-red)

1일차 강의 자료 · 8시간

워크 북 실습

D2_4.실습- Dual-Track UI + Logic TDD

04½

소프트웨어 개발(Development) 환경 구성

Python 실습 환경 · CS 필수 기반 · Cursor AI 프로젝트 구조 설정

4교시 전	사전 필수

소프트웨어 개발을 위해 알아야 할 것들 — CS 필수 기반

4교시 전

기초 CS 지식은 개발 역량의 "하한"과 "상한"을 결정합니다 — 언어·프레임워크는 바뀌어도 CS 기반은 영원합니다

1 자료구조 (Data Structures)

배열 · 리스트 · 스택 · 큐
해시맵 · 그래프 · 트리
→ 어떤 문제를 어떤 자료구조로 해결?

2 알고리즘 (Algorithms)

탐색 · 정렬 · 분할정복
DP · 그리디
Big-O 시간/공간 복잡도

3 운영체제 (OS)

프로세스/스레드
메모리 구조
동기화/경쟁 상태
→ async/await 이해

4 네트워크 (Networking)

TCP/IP · HTTP
DNS · REST
WebSocket
→ API 설계의 근본

5 데이터베이스 (Database)

SQL · 트랜잭션(ACID)
인덱스 · 조인 · 정규화
NoSQL 특성 (MongoDB, Redis)

본 실습은 Python 3.10+ 환경에서 진행합니다 — Java·JS 등 다른 언어로 구현하지 않도록 주의하세요

Python 실습 환경 구성 — GREEN 단계 착수 전 필수 확인

4교시 전

GREEN 단계를 시작하기 전 반드시 Python 환경과 프로젝트 구조를 확인하세요 — 잘못된 언어/환경에서 구현하면 테스트가 연결되지 않습니다

⚙️ 환경 체크리스트 (착수 전 필수)

☐ Python 버전 확인	python --version → 3.10 이상
☐ pytest 설치	pip install pytest → import 확인
☐ 프로젝트 루트 확인	Cursor에서 올바른 폴더를 열었는지 확인
☐ .cursorrules 설정	Python 언어 명시 룰 작성 완료
☐ 가상환경 활성화	venv 또는 conda env 활성 상태 확인
☐ 파일 연결 확인	test_*.py ↔ magic_square.py 같은 폴더

📁 올바른 프로젝트 구조 (Python)

```
magic_square_project/
├── magic_square.py       # 구현 (Entity+Control)
├── grid_ui.py            # UI   (Boundary)
├── test_logic_magic.py   # 로직 테스트
├── test_ui_magic.py      # UI   테스트
├── .cursorrules          # Cursor 규칙
└── requirements.txt      # pytest 등
```

📄 .cursorrules — Python 언어 명시

```
# .cursorrules
language: python
version: "3.10+"
test_framework: pytest
no_java: true   # Java 사용 금지
style: pep8
type_hints: required
```

⚠️ Java vs Python — 혼용 시 발생하는 문제

❌ Java로 구현	pytest가 .java 파일을 인식하지 못함 → 테스트 연결 불가
❌ 언어 혼용	import 경로·문법 충돌 → ModuleNotFoundError 발생
✅ Python으로 통일	test_*.py ↔ *.py 자동 연결 → pytest 즉시 실행 가능

Cursor AI에게 구현을 요청할 때 반드시 Python으로 작성하도록 프롬프트에 명시하세요 — "Python 3.10+, pytest 기반으로 구현해줘"

04½

PHASE 2 — [TRACK A] UI GREEN

최소 UX 구현 · 로직 주입 원칙 · Cursor Agent 활용

4교시

60분

4.1 UI GREEN — 로직을 주입(Inject)하라

4교시

"UI GREEN 단계에서 로직을 직접 쓰지 말고 주입(Inject)하라 — UI는 판단하지 않는다"

❌ UI가 판단하는 잘못된 구현

```
class GridUI:
  def submit(self):
    total = 0
    for row in self.grid:
        total += sum(row)
    if total == 34:  # 직접 판단!
        self.show("완성!")
    else:
        self.show("오류")

# ❌ UI가 비즈니스 규칙 포함
# ECB Boundary 원칙 위반
```

✅ 로직을 주입하는 올바른 구현

```
class GridUI:
  def __init__(self, validator):
    self.validator = validator  # 주입!
    self._visible = True
    self._error_visible = False

  def submit(self):
    # 판단은 validator에게!
    result = self.validator.is_valid(
        self.grid)
    self._show_result(result)

  def is_visible(self) -> bool:
    return self._visible  # UX만 담당
```

4.2 UI GREEN 실습 — GridUI 최소 구현 완성

4교시 실습

Cursor Agent 프롬프트: "@test_ui_magic.py 의 테스트를 통과하는 최소 GridUI 클래스를 구현해줘. 로직 판단 없이 UX만 담당"

grid_ui.py — UI GREEN 최소 구현

```python
# grid_ui.py  — ECB: <<boundary>>
from typing import Optional

class GridUI:
    """마방진 격자 UI — Boundary 계층. 판단하지 않고 전달만 한다."""

    def __init__(self, size: int, validator=None):
        self.size = size
        self.validator = validator
        self._visible: bool = True
        self._error_visible: bool = False
        self._result_visible: bool = False
        self._submit_enabled: bool = False
        self.grid: list = [[0]*size for _ in range(size)]

    def is_visible(self) -> bool:     return self._visible
    def error_visible(self) -> bool:  return self._error_visible
    def result_visible(self) -> bool: return self._result_visible
    def submit_enabled(self) -> bool: return self._submit_enabled

    def set_cell(self, row: int, col: int, value: int) -> None:
        self.grid[row][col] = value
    def cell_editable(self, row: int, col: int) -> bool:
        return self.grid[row][col] == 0  # 빈칸이면 편집 가능
```

pytest test_ui_magic.py → 모두 PASS → commit(feat: ui-green) → TRACK A GREEN 완료!

05

PHASE 2 — [TRACK B] Logic GREEN

최소 도메인 구현 · YAGNI · 하드코딩 허용 · Cursor Agent

5교시

60분

5.1 Logic GREEN — 테스트가 요구하는 것만 구현하라

5교시

"GREEN 단계의 완성 기준 = 테스트 100% 통과 — 하드코딩도 허용, 과잉 설계 금지(YAGNI)"

YAGNI = You Aren't Gonna Need It

원칙 1

테스트가 정의한 경우만

미래 예외 상황은
새 RED 테스트로 추가

원칙 2

일반화는 최소로 (YAGNI)

테스트가 요구하는 수준만
일반화 — 그 이상 금지

원칙 3

하드코딩 허용

값을 상수로 박아도 OK
REFACTOR에서 알고리즘으로

원칙 4

Cursor Agent 최대 활용

@test_logic_magic.py 제공
"이 테스트를 통과하는 최소 코드 작성해줘"

Logic GREEN은 UI를 모른다 — import grid_ui 절대 금지 · 도메인 로직만 집중

5.2 Magic Square Logic GREEN 구현 — 최소 도메인 코드

5교시 실습

Cursor Agent: "@test_logic_magic.py 테스트를 통과하는 최소 magic_square.py 작성. 타입힌트 필수. 하드코딩 허용."

magic_square.py — Logic GREEN (하드코딩 허용 단계)

```python
# magic_square.py  — ECB: <<entity>> + <<control>>
from typing import List

PRESET_4X4 = [[16,3,2,13],[5,10,11,8],[9,6,7,12],[4,15,14,1]]

class MagicSquare:
    """마방진 Entity + 검증 Control — ECB 혼합 (GREEN, REFACTOR에서 분리)"""

    def __init__(self, size: int) -> None:
        if size <= 0:
            raise ValueError(f"size는 양수여야 합니다: {size}")
        self.size = size
        self.grid: List[List[int]] = PRESET_4X4  # 하드코딩 OK

    def is_valid(self) -> bool:
        """모든 행/열/대각선 합이 magic_sum과 같은지 검증"""
        s = self.size * (self.size**2 + 1) // 2  # magic sum
        for i in range(self.size):
            if sum(self.grid[i]) != s: return False  # 행
            if sum(self.grid[r][i] for r in range(self.size)) != s: return False  # 열
        return True
```

코드 리뷰 6대 관점 (GREEN 후 Review)

① 코드 품질

- 변수/함수 명명 규칙
- 함수 길이 (20줄 이하)
- 들여쓰기 깊이

② 성능

- 불필요한 리렌더링
- N+1 쿼리 문제
- 비효율적 알고리즘

③ 타입 안정성

- any 타입 사용 여부
- 타입 가드 적용
- 제네릭 활용

④ 테스트 커버리지

- 엣지 케이스 테스트
- 에러 케이스 테스트
- 비동기 처리 테스트

⑤ 보안

- XSS 공격 방지
- SQL 인젝션 방지
- 입력 데이터 검증

⑥ 접근성 (a11y)

- aria-label 사용
- 키보드 네비게이션
- 색상 대비 기준

1일차 강의 자료 · 8시간

워크 북 실습

D2_5.실습- GREEN 단계 (테스트 통과)

06

PHASE 3 — 통합 REFACTOR

Lock-Step 동기화 · SOLID 적용 · 코드 스멜 제거 · 커버리지 80%+

6교시

60분

6.1 REFACTOR — "동작을 유지하면서 내부 구조를 개선한다"

6교시

리팩토링 = 외부 동작 유지 + 내부 구조 개선 — 기능 추가 ≠ 리팩토링 · 버그 수정 ≠ 리팩토링

"리팩토링 시작 전 3대 전제: ① 테스트 커버리지 80%+ ② Git 브랜치 분리 ③ 소규모 커밋"

Step 1: 코드 스멜 식별

- Mysterious Name 탐지
- Long Method (20줄+)
- Large Class (God Class)
- Duplicated Code

Step 2: SOLID 적용

- SRP: 책임 분리 (ECB)
- OCP: Strategy Pattern
- LSP: 인터페이스 재설계
- DIP: ABC 의존성 역전

Step 3: 검증 & 커밋

- pytest 전체 실행
- 커버리지 80%+ 유지
- flake8/pylint 점수
- git commit & push

"Lock-Step REFACTOR" — UI + Logic 동시 정리 · UX 행동 변화 금지 · 모든 테스트 GREEN 유지

6.1 REFACTOR — "동작을 유지하면서 내부 구조를 개선한다"

6교시

원칙과 지침

"Vision without execution is just a dream. Execution without vision is a nightmare."(비전 없는 실행은 악몽이고, 실행 없는 비전은 꿈일 뿐이다.)

넓게 바라보는 비전과 실행 가능한 준비가 동시에 필요하다는 것을 강조하는 경구입니다.

"Inspect and Adapt" (점검하고 적응하라)

넓게 관찰하고 현재 상황을 평가한 후, 실행 가능한 방법을 조정하여 개선해야 함을 강조합니다.

Agile 및 DevOps의 핵심 철학입니다.

리팩토링 시점 - Don Roberts의 3의 법칙

단계	상황	행동	예시
1단계	처음 발생	그냥 한다	새 기능 개발 시
2단계	두 번째 비슷한 일	중복 감수하고 진행	두 번째 같은 패턴
3단계	세 번째 동일 패턴	리팩토링 시작!	공통 함수 추출
쓰레기 줍기	코드 수정 중 발견	지나가면서 정리	Boy Scout Rule
이해 리팩토링	코드 파악 중	이해하며 정리	복잡한 함수 분리
변화 준비	새 기능 추가 전	추가 쉽게 구조 변경	OCP 원칙 적용

Refactoring은 언제 할까?

Before:

```
class OrderProcessor:
    def calculate_total(self, price: float, quantity: int, discount: float) -> float:
        total = 0

        if discount > 0:
            total = (price * quantity) - discount
        else:
            total = price * quantity

        # 불필요한 디버깅용 출력
        print(f"Total after discount: {total}")

        return total
```

이 코드의 문제점

print() 문은 디버깅용으로만 필요하고,
실제 서비스 코드에서는 로그 시스템으로
대체되거나 제거되어야 함.

if/else 문이 중복된 계산 로직(price * quantity)을
포함 → 가독성 저하.

"할인 적용 여부" 로직을 더 단순하게 표현할 수
있음.

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

- 코드베이스에 산재한 작고 명확한 문제들을 점진적으로 해결하여 코드의 품질을 서서히 향상시키는 것

예제 1: 불필요한 코드 제거-order_processor.py

After:

```
class OrderProcessor:
    def calculate_total(self, price: float, quantity: int, discount: float) -> float:
        total = price * quantity

        if discount > 0:
            total -= discount

        return total
```

변경 항목	설명
디버깅 코드 제거	print() 문 제거 → 실제 서비스 코드에서는 불필요함
조건문 단순화	if discount > 0 블록만 남기고, 중복된 계산 제거
가독성 향상	한눈에 "가격 × 수량 → 할인 차감 → 반환" 흐름이 보임
유지보수 용이성 향상	나중에 로깅 시스템(logging)이나 추가 정책을 쉽게 붙일 수 있음

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 2: 중복 코드 정리

Before:

```
def send_email_to_user(user_email):
    print(f"Sending email to user: {user_email}")

def send_email_to_admin(admin_email):
    print(f"Sending email to admin: {admin_email}")
```

문제점:
두 함수 모두 이메일을 보내는 기능이 동일한데,
수신자 종류(user, admin)만 다름 → 중복 코드 발생

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 2: 중복 코드 정리

After:

```
class EmailSender:
    def send_email(self, email, recipient_type):
        print(f"Sending email to {recipient_type}: {email}")
```

개선점:

- send_email_to_user()와 send_email_to_admin()을 하나로 통합
- 수신자 구분은 recipient_type 인자("user", "admin")로 처리
- 코드 중복 제거, 유지보수 용이

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 3: 의미 없는 변수 및 로직 제거

Before:

```
class AverageCalculator:
    def calculate_average(self, numbers):
        total_sum = 0
        for number in numbers:
            total_sum += number

        total_count = len(numbers)
        average_value = float(total_sum) / total_count  # 불필요한 중간 변수
        return average_value
```

문제점:

average_value 변수는 한 번만 사용되고, 의미가 중복됨

단순 계산 결과를 저장하기 때문에 불필요한 변수 선언

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 3: 의미 없는 변수 및 로직 제거

After:

```
class AverageCalculator:
    def calculate_average(self, numbers):
        total_sum = 0
        for number in numbers:
            total_sum += number
        return total_sum / len(numbers)
```

개선점:

- 불필요한 변수 average_value 제거
- 계산식을 return 문에서 바로 표현하여 더 간결하고 직관적
- 코드의 가독성, 유지보수성 향상

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 4. 사용되지 않는 코드 제거

Before:

```
class PriceCalculator:
    def calculate_total_price(self, price, quantity):
        return price * quantity

    def unused_function(self):
        print("This function is not used.")
```

문제점:

unused_function()은 프로그램 어디에서도 호출되지 않음

의미 없는 코드가 유지보수성과 가독성을 떨어뜨림

이런 함수는 “죽은 코드(dead code)”로 간주됨

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 4. 사용되지 않는 코드 제거

After:

```
class PriceCalculator:
    def calculate_total_price(self, price, quantity):
        return price * quantity
```

개선점:

사용되지 않는 unused_function() 메서드를 삭제

코드가 더 간결하고 깨끗하게(clean code) 됨

클래스의 책임이 명확해지고, 유지보수성이 향상됨

Refactoring은 언제 할까?

"이해하기 위한 리팩토링 (Refactoring to Understand)"

Before:

```
class CommissionCalculator:
    def a(self, x, y):
        return x * 0.05 * y / (x + y)
```

문제점:

함수명 a, 변수명 x, y는 의미가 불분명합니다.

코드의 의도("판매 수수료율 계산")를 한눈에 파악하기 어렵습니다.

유지보수 시 다른 개발자가 이해하기 힘듭니다.

Refactoring은 언제 할까?

"이해하기 위한 리팩토링 (Refactoring to Understand)"

After:

```
class CommissionCalculator:
    def calculate_commission_rate(self, sales, expenses):
        return sales * 0.05 * expenses / (sales + expenses)
```

개선점:

a() → calculate_commission_rate()로 변경 → 기능 목적이 명확

변수 x, y → sales, expenses로 의미 부여

코드의 계산 구조를 그대로 유지하면서 이해하기 쉬운 형태로 개선

Refactoring은 언제 할까?

"변화를 위한 리팩토링 (Refactoring to Change)"

Before:

```
class OrderProcessor:
    def process_order(self, order_type):
        if order_type == "online":
            print("Processing online order")
        elif order_type == "in_store":
            print("Processing in-store order")
        else:
            print("Unknown order type")
```

문제점:

새로운 주문 유형("subscription" 등)이 추가될 때마다
if-elif 조건문을 계속 추가해야 함 → OCP(Open-Closed Principle) 위반

코드 수정 범위가 넓고, 유지보수성이 낮음

Refactoring은 언제 할까?

"변화를 위한 리팩토링 (Refactoring to Change)"

After:

```
class OrderProcessor:
    def __init__(self):
        self.order_processors = {
            "online": self._process_online_order,
            "in_store": self._process_in_store_order
        }

    def process_order(self, order_type):
        processor = self.order_processors.get(order_type)
        if processor:
            processor()
        else:
            print("Unknown order type")

    def _process_online_order(self):
        print("Processing online order")

    def _process_in_store_order(self):
        print("Processing in-store order")
```

개선점:

주문 유형별 처리 로직을 딕셔너리(Map)로 등록

새로운 주문 유형이 생기면 단순히 self.order_processors에 추가만 하면 됨

조건문 제거, 확장에 유연한 구조

코드가 OCP(Open-Closed Principle)에 부합

코드 스멜 (Code Smell) 7가지 유형

Mysterious Name

- 모호한 이름 → 명확한 이름으로
- calc() → calculate_discounted_price()
- 리팩토링: Change Function Declaration

Duplicated Code

- 같은 코드 2곳 이상 존재
- 공통 함수로 추출
- 리팩토링: Extract Function

Long Method

- 20줄 초과 함수
- 관련 코드를 함수로 분리
- 리팩토링: Extract Method

Large Class

- 너무 많은 책임
- SRP 위반 클래스
- 리팩토링: Extract Class

코드 스멜 (Code Smell) 7가지 유형

Feature Envy

- 다른 클래스 데이터 과도 접근
- Invoice가 Address 데이터 직접 조작
- 리팩토링: Move Method

Divergent Change

- 다양한 이유로 자주 수정
- 여러 책임이 한 클래스에 혼재
- 리팩토링: Extract Class

Shotgun Surgery

- 수정 시 여러 클래스를 동시에 고침
- 산탄총처럼 영향 범위 분산
- 리팩토링: Move Field, Combine Functions

6.2 Magic Square REFACTOR — SRP 적용: Entity + Control 분리

6교시 실습

GREEN 단계의 하드코딩 → REFACTOR: 알고리즘 적용 + ECB 책임 분리 (SRP)

❌ Before: 하드코딩 + 책임 혼재

```python
class MagicSquare:
    # Entity + Control 혼재 ← SRP 위반
    def __init__(self, size: int):
        self.size = size
        self.grid = PRESET_4X4  # 하드코딩!

    def is_valid(self) -> bool: ...
    def find_missing(self): ...
    def generate_siamese(self): ...
    # 너무 많은 책임!
```

✅ After: SRP 적용 → 책임 분리

```python
class MagicSquare:  # <<entity>>
    """격자 데이터만 담당"""
    def __init__(self, size: int):
        self.size = size
        self.grid = self._generate()

    def _generate(self) -> list:
        # Siamese 알고리즘
        n = self.size
        grid = [[0]*n for _ in range(n)]
        ...

class SquareValidator:  # <<control>>
    """검증 로직만 담당"""
    def is_valid(self, sq: MagicSquare):
        s = sq.size*(sq.size**2+1)//2
        ...
```

6.3 Dual-Track Lock-Step REFACTOR — 5단계 전체 흐름

6교시

0 STEP 0

입력물 준비

PRD · UX Contract · Logic Rule
⚠️ 이 단계에서는 코드 금지

→

1 STEP 1

UI/UX RED

디자인 기반 실패 UI 테스트
UX Contract 선언

→

2 STEP 2

Logic RED

도메인 규칙 기반 실패 테스트
Gherkin→pytest 변환

→

3 STEP 3

Logic GREEN

UI 고려 없이 최소 도메인 구현
하드코딩 허용

→

4 STEP 4

UI GREEN

로직 주입(Inject)
UX 테스트 통과만 목표

→

5 STEP 5

REFACTOR

UI + Logic 동시 정리
Lock-Step 동기화

1일차 강의 자료 · 8시간

워크 북 실습

D2_6.실습- Refactoring 단계
D2_7실습- Dual-Track Refactoring

07

TRACEABILITY — 4계층 추적성 검증

Invariant → UI Contract Test → Domain Test → Implementation Unit

7교시

60분

7.1 Traceability 4계층 — 의미가 코드까지 소실되지 않도록

7교시

Concept-to-Code Traceability: 개념→책임→테스트→코드 각 계층의 연결고리가 끊어지지 않아야 한다

Layer 1 — **INVARIANT (불변 조건)**

"무엇이 절대 바뀌면 안 되는가"

마방진의 본질적 규칙
→ 행/열/대각선 합 = 34
→ 숫자 1~16 각 1번씩

수학적 진실

↓

Layer 2 — **UI CONTRACT TEST (경계 계층)**

"UX가 올바르게 반응하는가"

GridUI 테스트
→ 그리드가 보인다
→ 빈칸이 편집 가능
→ 오류 메시지 안 보임

Boundary 검증

↓

Layer 3 — **DOMAIN TEST (제어 계층)**

"비즈니스 규칙이 작동하는가"

SquareValidator 테스트
→ is_valid() True/False
→ find_missing() 좌표
→ MissingError 발생

Control 검증

↓

Layer 4 — **IMPLEMENTATION UNIT (구현 단위)**

"알고리즘이 정확한가"

단위 테스트
→ Siamese 알고리즘
→ 합 계산 정확성
→ 범위 검증 로직

Entity 검증

검증 방향: Invariant(수학 진실) → UI Contract(경계 계약) → Domain(비즈니스 규칙) → Unit(구현 단위)

7.2 Traceability 매트릭스 — Magic Square 전체 추적 검증

7교시 실습

모든 테스트가 어느 계층·개념·시나리오와 연결되는지 추적 — "테스트가 없는 코드는 제거 대상"

테스트 ID	계층	ECB	시나리오	테스트 이름	상태
T-UI-01	Layer2	Boundary	L1	`test_grid_is_displayed`	●
T-UI-02	Layer2	Boundary	L2	`test_blank_cell_is_editable`	●
T-UI-03	Layer2	Boundary	L3	`test_error_msg_hidden`	●
T-DOM-01	Layer3	Control	L1	`test_valid_4x4_magic_square`	●
T-DOM-02	Layer3	Control	L3	`test_invalid_row_sum`	●
T-DOM-03	Layer3	Control	L3	`test_missing_zero_blanks`	●
T-UNIT-01	Layer4	Entity	L2	`test_siamese_algorithm`	●
T-UNIT-02	Layer4	Entity	L3	`test_size_zero_raises`	●

`pytest --cov=. --cov-report=term-missing` → 모든 테스트 PASS + Coverage 80%+ 확인

실습: 위 매트릭스의 누락 테스트를 찾아 추가하고 coverage gap을 줄여보세요

08

Prompting Best Practices (BP) 발표

팀별 BP 공유 · KPT 회고 · SMART 목표 설정 · 2일 과정 마무리

8교시

60분

8.1 AI-Safe Prompting — 5대 황금 원칙

8교시

"Cursor AI는 구현 보조 도구다 — 설계는 개발자가, 테스트는 먼저, 검증은 반드시"

BP 1 실패 조건(RED) 선언 먼저

테스트 코드로 실패 조건 명확히 정의
→ AI가 충족하는 코드 생성
"무엇이 틀렸는지"를 먼저 결정

BP 2 계약 중심(Contract-First)

함수 시그니처·입출력·예외 조건 먼저
→ AI에게 @파일 참조로 계약 전달
인터페이스부터 정의 후 구현 요청

BP 3 단계별 Lock-Step

한 번에 하나씩 작은 단위로
→ 너무 많은 것은 AI 품질 저하
RED→GREEN→REFACTOR 순서 유지

BP 4 컨텍스트 충분히 제공

@파일명으로 기존 코드·아키텍처 참조
.cursorrules로 프로젝트 규칙 고정
ECB 계층 명시 ("이것은 <<control>>")

BP 5 반드시 테스트로 검증

AI가 생성한 코드 = 신뢰 불가
→ 항상 pytest로 검증
테스트 없는 AI 코드 = 기술 부채

Sprint Retrospective 개요

- 목적: 지난 스프린트에서 잘한 점/개선할 점을 공유하고 다음 이터레이션에 반영
- 시기: 이터레이션 종료 시점 (매 Sprint 끝), 릴리즈 후에도 가능
- 참여자: 팀원, 전문가, 팀 리더, 제품 책임자(PO)
- 산출물: 구체적인 개선 액션 아이템 (다음 Sprint에 반영)
- 시간 제한(Time Box): 전체 1.5-2시간 이내
- 원칙: 비난 없는 분위기, 개방적 대화, 팀 전체 관점

애자일 회고 5단계 절차

단계	활동	Time Box	대표 기법
① 체크인/준비	분위기 조성, 참여자 집중 유도	5-10분	ESVP, 한 마디 소감
② 데이터 수집	이터레이션 동안 일어난 일 모으기	20-30분	좋았/나빴던 점, 학습 매트릭스
③ 통찰 이끌기	패턴 파악, 근본 원인 분석	20분	점 투표, Fishbone Diagram
④ 결정하기	개선 액션 아이템 결정	20분	SMART 목표, EXACT 기준
⑤ 마무리	회고 자체에 대한 피드백	10분	+/Delta, Follower 구하기

회고 기법 - KPT & 학습 매트릭스

K - Keep (유지)

- 잘 됐던 것, 계속 유지할 것
- TDD 적용으로 버그 감소
- Daily Standup 15분 유지

P - Problem (문제)

- 문제였던 것, 개선 필요한 것
- 테스트 커버리지 60% 미달
- PR 리뷰 48시간 초과

T - Try (시도)

- 다음에 시도할 것
- Pair Programming 도입
- flake8 CI 게이팅 추가

학습 매트릭스

- + 잘된 점
- - 나빴던 점
- C 고칠 내용
- ? 불확실한 점

ESVP 체크인

- E: 탐험가 (적극 참여)
- S: 쇼핑객 (유용한 것만)
- V: 관광객 (그냥 참여)
- P: 죄수 (강제 참여)

Fishbone Diagram

- 근본 원인 분석 도구
- 결과(Effect)를 중심으로
- 원인(Cause) 가지 분석

회고 개선 목표 - SMART 원칙

원칙	의미	나쁜 예	좋은 예
S - Specific	구체적	테스트 더 쓰기	단위 테스트 10개 추가
M - Measurable	측정 가능	코드 개선	커버리지 80%→85%
A - Attainable	달성 가능	버그 0	High 버그 50% 감소
R - Realistic	현실적	기술부채 전부 해결	핵심 모듈 리팩토링 3개
T - Time-based	시간 제한	언젠가 개선	다음 Sprint(2주) 내 완료

8.3 BP 발표 형식 & 팀 공유 세션 진행

8교시 실습

📋 팀별 BP 발표 구조 (5분)

① **BP 제목**
한 줄로 핵심을 표현
예: "RED 단계에서 시나리오 수준 분리"

② **상황 (Context)**
언제 이 원칙이 필요한가?
예: "Logic RED 작성 시 L2/L3 혼재 문제"

③ **해결책 (Solution)**
구체적인 프롬프트 또는 방법
예: Gherkin Level 표시 추가

④ **효과 (Effect)**
적용 전/후 비교
예: 테스트 누락 0건, 리뷰 30% 단축

⑤ **팀 적용 제안**
다음 프로젝트에 어떻게 적용?
.cursorrules 반영 여부

🔄 KPT 회고 + SMART 목표

K — Keep

"TDD RED 먼저 작성"
"Cursor Ask → Plan → Agent 순서"
"ECB 계층 주석 명시"

P — Problem

"GREEN에서 과도한 구현 시도"
"UI/Logic 경계 혼재"
"커버리지 미확인 커밋"

T — Try

"Work Order 형식 .cursorrules 등록"
"BP 체크리스트 PR 게이팅"
"매 sprint KPT 15분 진행"

SMART 목표: 구체적·측정 가능·달성 가능·현실적·시간 제한

1일차 강의 자료 · 8시간

워크 북 실습

D2_8.실습- Prompting BP

2일차 핵심 정리

01

To-Do → Work Order + Cursor AI 협업

판단 포함 to-do만 테스트 변환 · Work Order = 계약 중심 · 5대 BP 원칙 적용
Cursor 모드: RED(Ask/Plan) → GREEN(Agent) → REFACTOR(Agent+Plan)

02

PHASE 1 — UI RED & Logic RED (동시)

UI Track: UX Contract 언어(보인다/허용) → GridUI 실패 선언 · Logic Track: Gherkin L0~L3 → pytest 변환
⚠️ 두 트랙 완전 독립 · UI는 Logic을 모른다 · 모두 FAIL = 정상

03

PHASE 2 — UI GREEN & Logic GREEN (최소)

Logic GREEN: 하드코딩 허용 · YAGNI · 테스트 통과만 목표 · UI GREEN: 로직 주입(Inject) · 판단 없음
완성 기준 = 테스트 100% PASS · Cursor Agent로 구현 자동화

04

PHASE 3 + TRACEABILITY + BP 발표

Lock-Step REFACTOR: UI+Logic 동시 · SOLID · 스멜 제거 · 커버리지 80%+
4계층 추적: Invariant→UI Contract→Domain→Unit · KPT 회고 · SMART 목표

"앞으로의 엔지니어는 단순한 코더가 아니라, AI와 협업하여 문제를 해결하는 지식 설계자입니다" — 김대경 Ph.D.

D2_실습 과제 전체 가이드

[해결할 문제]

- 마방진(Magic Square)이란 가로, 세로, 대각선 방향의 수를 더한 값이 모두 같은 정사각형 행렬입니다. 마방진 에는 `1`부터 `정사각형 넓이`까지, 수가 하나씩 배치되어야 합니다. 아래는 가로, 세로, 대각선 방향의 수를 더한 값이 모두 34인 4 x 4 마방진입니다.

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

1. 두 빈칸의 위치를 찾습니다.
2. 숫자 1 ~ 16 중 존재하지 않는 숫자 2개를 찾습니다.
3. 첫 번째 빈칸에 작은 숫자를, 두 번째 빈칸에 큰 숫자를 넣어 행렬이 마방진이 되는 지 검사합니다.
4. return

4-1. 마방진이라면 [작은 숫자의 행 번호, 작은 숫자의 열 번호, 작은 숫자, 큰 숫자의 행 번호, 큰 숫자의 열 번호, 큰 숫자]를 return 합니다.

4-2. 마방진이 아니라면 [큰 숫자의 행 번호, 큰 숫자의 열 번호, 큰 숫자, 작은 숫자의 행 번호, 작은 숫자의 열 번호, 작은 숫자]를 return 합니다.

D2_실습 과제 전체 가이드

```java
package com.samsung;

import java.util.ArrayList;
import java.util.Arrays;

class Pair {
    Integer y;
    Integer x;

    public Pair(Integer y, Integer x) {
        this.y = y;
        this.x = x;
    }

    public Integer first() {
        return y;
    }

    public Integer second() {
        return x;
    }
}
```

```java
class MagicSquare {
    public static final int n = 4;

    // 빈칸(0)의 좌표를 찾는 메서드
    public ArrayList<Pair> find_blank_coords(int[][] matrix) {
        ArrayList<Pair> coords = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 0) {
                    coords.add(new Pair(i, j));
                }
            }
        }
        return coords;
    }

...
```

D2_실습 과제 전체 가이드

```java
// 존재하지 않는 숫자를 찾는 메서드
public ArrayList<Integer> find_not_exist_nums(int[][] matrix) {
    ArrayList<Integer> nums = new ArrayList<>();
    boolean[] exist = new boolean[n * n + 1];
    Arrays.fill(exist, false);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] != 0) {
                exist[matrix[i][j]] = true;
            }
        }
    }

    for (int i = 1; i <= n * n; i++) {
        if (!exist[i]) {
            nums.add(i);
        }
    }

    return nums;
}
```

```java
// 마방진인지 확인하는 메서드
public boolean is_magic_square(int[][] matrix) {
    int sum = n * (n * n + 1) / 2; // 마방진의 합

    // 행, 열 합 확인
    for (int i = 0; i < n; i++) {
        int row_sum = 0, col_sum = 0;
        for (int j = 0; j < n; j++) {
            row_sum += matrix[i][j];
            col_sum += matrix[j][i];
        }
        if (row_sum != sum || col_sum != sum) {
            return false;
        }
    }

    // 대각선 합 확인
    int main_diagonal_sum = 0, skew_diagonal_sum = 0;
    for (int i = 0; i < n; i++) {
        main_diagonal_sum += matrix[i][i];
        skew_diagonal_sum += matrix[i][n - 1 - i];
    }

    return main_diagonal_sum == sum && skew_diagonal_sum == sum;
}
```

D2_실습 과제 전체 가이드

```java
// 마방진이 되는지 확인하는 솔루션 메서드
  public int[] solution(int[][] matrix) {
    int[] answer = new int[6];
    int ans_idx = 0;

    // 1. 빈칸(0)의 좌표 찾기
    ArrayList<Pair> coords = find_blank_coords(matrix);

    // 2. 존재하지 않는 숫자 찾기
    ArrayList<Integer> nums = find_not_exist_nums(matrix);
    nums.sort(Integer::compareTo); // 작은 숫자 먼저

    // 3. 첫 번째 조합: 작은 숫자 -> 첫 번째 빈칸, 큰 숫자 -> 두 번째 빈칸
    matrix[coords.get(0).first()][coords.get(0).second()] = nums.get(0);
    matrix[coords.get(1).first()][coords.get(1).second()] = nums.get(1);

    if (is_magic_square(matrix)) {
        // 마방진이 되는 경우
        for (int i = 0; i < 2; i++) {
          answer[ans_idx++] = coords.get(i).first() + 1; // 행 번호 (1-indexed)
          answer[ans_idx++] = coords.get(i).second() + 1; // 열 번호 (1-indexed)
          answer[ans_idx++] = nums.get(i); // 숫자
        }
      } else {
        // 4. 두 번째 조합: 큰 숫자 -> 첫 번째 빈칸, 작은 숫자 -> 두 번째 빈칸
        matrix[coords.get(0).first()][coords.get(0).second()] = nums.get(1);
        matrix[coords.get(1).first()][coords.get(1).second()] = nums.get(0);
        for (int i = 0; i < 2; i++) {
          answer[ans_idx++] = coords.get(1 - i).first() + 1; // 행 번호 (1-indexed)
          answer[ans_idx++] = coords.get(1 - i).second() + 1; // 열 번호 (1-indexed)
          answer[ans_idx++] = nums.get(i); // 숫자
        }
      }

      return answer;
  }
}
```

D2_실습 과제 전체 가이드

```java
public class MagicSquareMain {
    public static void main(String[] args) {
        MagicSquare magicSquare = new MagicSquare();

        // 입력 행렬
        int[][] matrix = {
            {16, 2, 3, 13},
            {5, 11, 10, 0},
            {9, 7, 6, 12},
            {0, 14, 15, 1}
        };

        // 결과 출력
        int[] ret = magicSquare.solution(matrix);
        System.out.println("결과 값은 " + Arrays.toString(ret) + " 입니다.");
    }
}
```

D2_실습 과제 전체 가이드

0으로 표시된 위치와 채워져야 하는 값을 리턴합니다.

matrix	return
[[16,2,3,13],[5,11,10,0],[9,7,6,12],[0,14,15,1]]	[4,1,4,2,4,8]

D2_실습 과제 전체 가이드

1. 테스트 코드 작성

(1) find_blank_coords() 메서드 테스트

- 목표: 행렬에서 0 값을 가진 좌표를 정확히 반환하는지 검증.
- 테스트 항목:
  - 빈칸이 여러 개일 때 올바른 좌표 반환.
  - 빈칸이 없는 경우 빈 리스트 반환.

(2) find_not_exist_nums() 메서드 테스트

- 목표: 1~16의 숫자 중 행렬에 없는 숫자를 올바르게 반환하는지 검증.
- 테스트 항목:
  - 일부 숫자가 누락된 경우 올바른 숫자 리스트 반환.
  - 모든 숫자가 존재하는 경우 빈 리스트 반환.

(3) is_magic_square() 메서드 테스트

- 목표: 행렬이 마방진인지 여부를 정확히 판단하는지 검증.
- 테스트 항목:
  - 가로, 세로, 대각선 합이 모두 같은 경우 true 반환.
  - 합이 일치하지 않는 경우 false 반환.

(4) solution() 메서드 테스트

- 목표: 두 개의 빈칸에 숫자를 넣어 마방진을 완성하거나 실패하는 결과를 올바르게 반환.
- 테스트 항목:
  - 빈칸이 2개이며 숫자를 채웠을 때 마방진이 완성되는 경우.
  - 숫자를 어떤 방식으로 채워도 마방진이 되지 않는 경우.
  - 빈칸이 없는 경우.
  - 행렬이 이미 마방진인 경우.

D2_실습 과제 전체 가이드

JUnit 기반 테스트 코드 예시

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.Arrays;

class MagicSquareTest {

    MagicSquare magicSquare = new MagicSquare();

    @Test
    void testFindBlankCoords() {
        int[][] matrix = {
            {16, 2, 3, 13},
            {5, 11, 10, 0},
            {9, 7, 6, 12},
            {0, 14, 15, 1}
        };
        ArrayList<Pair> result = magicSquare.find_blank_coords(matrix);
        assertEquals(2, result.size());
        assertEquals(new Pair(1, 3).first(), result.get(0).first());
        assertEquals(new Pair(1, 3).second(), result.get(0).second());
        assertEquals(new Pair(3, 0).first(), result.get(1).first());
        assertEquals(new Pair(3, 0).second(), result.get(1).second());
    }

    @Test
    void testFindNotExistNums() {
        int[][] matrix = {
            {16, 2, 3, 13},
            {5, 11, 10, 8},
            {9, 7, 6, 12},
            {4, 14, 15, 1}
        };

        ArrayList<Integer> result = magicSquare.find_not_exist_nums(matrix);
        assertEquals(Arrays.asList(8), result);
    }
```

D2_실습 과제 전체 가이드

JUnit 기반 테스트 코드 예시

```java
@Test
  void testIsMagicSquare() {
    int[][] magicMatrix = {
      {16, 2, 3, 13},
      {5, 11, 10, 8},
      {9, 7, 6, 12},
      {4, 14, 15, 1}
    };

    assertTrue(magicSquare.is_magic_square(magicMatrix));

    int[][] nonMagicMatrix = {
      {16, 2, 3, 13},
      {5, 11, 10, 0},
      {9, 7, 6, 12},
      {0, 14, 15, 1}
    };

    assertFalse(magicSquare.is_magic_square(nonMagicMatrix));
  }
```

```java
@Test
  void testSolution() {
    int[][] matrix = {
      {16, 2, 3, 13},
      {5, 11, 10, 0},
      {9, 7, 6, 12},
      {0, 14, 15, 1}
    };

    int[] result = magicSquare.solution(matrix);
    assertEquals(6, result.length);
    assertArrayEquals(new int[]{2, 4, 8, 4, 1, 4}, result);
  }
}
```

D2_실습 과제 전체 가이드

2. 리팩토링 항목

(1) Pair 클래스

- 현재 Pair 클래스는 행렬의 좌표를 표현하기 위해 사용됩니다.
- 문제점:
  - 클래스 이름이 좌표를 명확히 표현하지 않음.
  - 메서드 이름(first()와 second())이 의미가 모호함.
- 리팩토링 방안:
  - 클래스 이름을 Coordinate로 변경.
  - 메서드 이름을 getRow()와 getColumn()으로 변경하여 명확히 표현.

D2_실습 과제 전체 가이드

2. 리팩토링 항목

(1) Pair 클래스

```
class Coordinate {
    private Integer row;
    private Integer col;

    public Coordinate(Integer row, Integer col) {
        this.row = row;
        this.col = col;
    }

    public Integer getRow() {
        return row;
    }

    public Integer getColumn() {
        return col;
    }
}
```

D2_실습 과제 전체 가이드

2. 리팩토링 항목

(2) 상수화

- n = 4와 같은 매직 넘버를 사용하고 있습니다.
- 리팩토링 방안:
  - n을 MATRIX_SIZE로 상수화하여 의미를 명확히 합니다.

```
public static final int MATRIX_SIZE = 4;
```

D2_실습 과제 전체 가이드

2. 리팩토링 항목

(3) 반복 코드 제거

•행렬의 합을 계산하는 코드에서 행과 열의 합을 각각 계산하는 반복문이 중복됩니다.
•리팩토링 방안:

- 행과 열 합 계산 로직을 메서드로 추출.

D2_실습 과제 전체 가이드

2. 리팩토링 항목

(3) 반복 코드 제거

```
private boolean checkSum(int[][] matrix, int sum) {
    for (int i = 0; i < MATRIX_SIZE; i++) {
        int rowSum = 0, colSum = 0;
        for (int j = 0; j < MATRIX_SIZE; j++) {
            rowSum += matrix[i][j];
            colSum += matrix[j][i];
        }
        if (rowSum != sum || colSum != sum) {
            return false;
        }
    }
    return true;
}
```

D2_실습 과제 전체 가이드

프로젝트 : Magic Square

본인 번호로 Repository 생성, 구현 진행

- Repository : 프로젝트명_개인번호 (ex: Magic_Square _xx)
- description : Author, Reviewer 성명
  - ex: Author : 홍길동, Reviewer : 다자바
- dev branc를 생성 하여 진행
- Commit : 상세한 commit
- Push/Pull Request/Review 진행하세요.

실습 평가 기준 (Magic Square 프로젝트)

평가 항목	배점	세부 기준	도구
테스트 (TDD)	40점	RED→GREEN→REFACTOR 사이클 준수	pytest
코드 품질	30점	SOLID, 클린코드, 스멜 제거	pylint
PRD/Gherkin	20점	시나리오 완성도, 추적성	문서
Git 관리	10점	커밋 단위, 메시지, 브랜치	GitHub
보너스	+10점	CI/CD GitHub Actions 구성	Actions

테스트 자동화 ROI 분석

+15~35%

TDD 개발시간
증가

-40~90%

결함률
감소

-50%

유지보수
비용 감소

-60%

버그 수정
비용 절감

-40%

온보딩
시간 단축

-30%

코드 리뷰
시간 절감

출처: Microsoft Research / IBM Systems Sciences Institute

TDD 실습 시 자주 겪는 문제와 해결법

- ImportError: 모듈 없음 → PYTHONPATH 설정 또는 conftest.py에 sys.path 추가
- 테스트가 항상 통과: 항상 True assertion 여부 확인, RED 먼저 확인
- Mock이 작동 안 함: unittest.mock.patch 경로 확인 (import된 경로 기준)
- 커버리지가 낮음: --cov-branch 옵션 추가, 엣지 케이스 테스트 보완
- GREEN 단계에서 과도한 구현: '지금 실패하는 테스트만' 원칙으로 복귀
- REFACTOR 후 테스트 실패: 기능 변경 금지 원칙 재확인, git diff로 검토
- flake8 오류 폭증: .flake8 설정 파일에 max-line-length=120 등 조정

실습 최종 과제 - Magic Square 완성 요구사항

❌ Bad Code

```
# 제출 구조
magic_square_project/
├── magic_square.py   # 구현 코드
├── test_magic.py     # pytest 테스트
├── PRD.md            # Gherkin 시나리오
├── ECB_diagram.md    # ECB 모델
├── .cursorrules      # AI 규칙
├── .github/
│   └── workflows/
│       └── ci.yml    # GitHub Actions
└── README.md         # 실행 방법
```

✅ Good Code

```
# 평가 기준
✅ TDD (40점)
   - RED 실패 테스트 5개 이상
   - GREEN 모두 통과
   - REFACTOR 1개 이상

✅ 코드 품질 (30점)
   - pylint 8.0 이상
   - mypy 오류 0
   - 함수 20줄 이하

✅ PRD/Gherkin (20점)
   - 3개 이상 시나리오
   - Given/When/Then 형식

✅ Git (10점)
   - 의미있는 커밋 메시지
   - 브랜치 전략 사용
```

D2 핵심 메시지 4가지

01
테스트 없으면 코드 없다

RED를 먼저 써야 GREEN이 의미있다

03
리팩토링은 기능 추가가 아니다

외부 동작 유지 + 내부 구조 개선 = 리팩토링

02
동작하게, 올바르게, 빠르게

GREEN → SOLID 원칙 → 성능 최적화 순서

04
AI와 협력하여 설계하라

Cursor AI는 구현 보조 도구, 설계는 개발자가

AI 코딩 시대의 개발자 역할 변화

이전 개발자의 역할

- 구문 암기 (syntax memory)
- 반복적 보일러플레이트 작성
- 문서 검색 + 코드 복붙
- 혼자서 전체 구현
- 단순 기능 구현에 시간 소비

AI 시대 개발자의 역할

- 문제 정의와 설계 (Problem Design)
- 테스트 기반 요구사항 명세
- AI 출력물 검증과 리팩토링
- 팀과 AI의 협업 조율
- 비즈니스 가치 창출에 집중

자주 묻는 질문 (FAQ)

- Q. TDD가 개발 속도를 늦추지 않나요?

  A. 초기 10-20% 느리나, 버그 수정 50% 절감 → 장기적으로 빠름
- Q. 커버리지 100%가 목표인가요?

  A. 80% 이상이 현실적 목표. 100%는 비효율적이며 의미 없는 테스트 양산
- Q. Cursor AI 없이도 TDD 가능한가요?

  A. 물론 가능. Cursor는 GREEN 단계 구현 속도를 높이는 보조 도구
- Q. 리팩토링 중 기능이 바뀌면?

  A. 즉시 중단 → 기능 변경은 새 RED 사이클로 분리
- Q. 프론트엔드에서도 TDD 적용되나요?

  A. Jest + React Testing Library로 UI Track TDD 가능

참고 자료 & 도서

- 📚 Clean Code - Robert C. Martin (클린 코드의 바이블)
- 📚 Refactoring 2nd Edition - Martin Fowler (리팩토링 대백과)
- 📚 Test-Driven Development - Kent Beck (TDD 창시자 저서)
- 📚 Working Effectively with Legacy Code - Michael Feathers
- 🌐 pytest 공식 문서: https://docs.pytest.org
- 🌐 Cursor AI 문서: https://docs.cursor.com
- 🌐 Anthropic Prompt Engineering: https://docs.anthropic.com
- 🌐 Python PEP8 스타일 가이드: https://peps.python.org/pep-0008

THANK YOU

앞으로의 엔지니어는
단순한 '코더'가 아니라
AI와 협력하여 문제를 해결하는
지식 설계자입니다.